Exercice 3 : Gestion d'un personnel universitaire

Rappel de l'énoncé

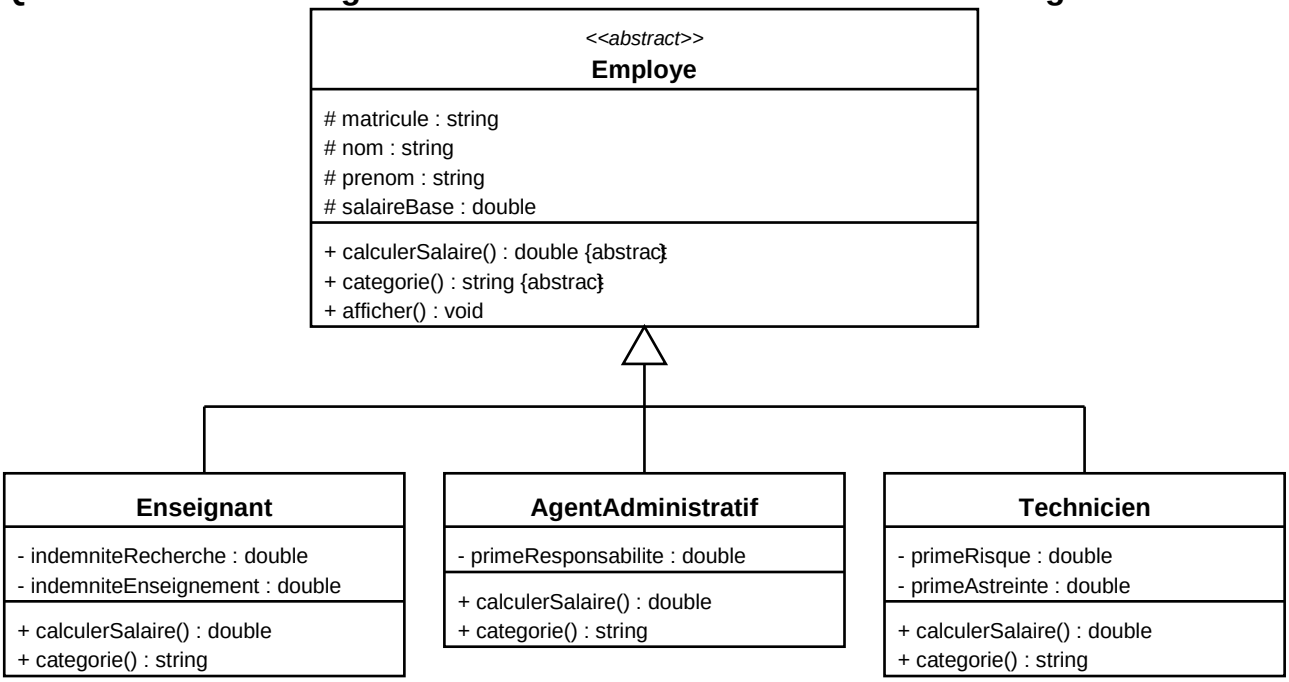
Une université possède plusieurs catégories de personnel : enseignants, agents administratifs et techniciens. Tous les employés possèdent un matricule, un nom, un prénom et un salaire de base. Cependant, les enseignants perçoivent des indemnités spécifiques, tandis que les agents administratifs et les techniciens bénéficient d'autres primes.

Question 1 : Identifier la classe mère et les classes dérivées

- **Classe mère : Employe**. Elle regroupe les attributs communs à tous les employés : matricule, nom, prénom, salaire de base.
- **Enseignant** : indemnité de recherche et indemnité d'enseignement.
- **AgentAdministratif** : prime de responsabilité.
- **Technicien** : prime de risque et prime d'astreinte.

Employe est une classe abstraite : la méthode calculerSalaire() est virtuelle pure, car un employé générique n'a pas de règle de calcul propre. On ne peut donc pas l'instancier. Chaque classe dérivée fournit sa propre formule.

Questions 2 et 3 : Diagramme de classes UML et relations d'héritage



Les trois flèches à triangle creux représentent l'héritage (relation « est un ») : un Enseignant, un AgentAdministratif et un Technicien sont des Employe. Le signe # désigne un attribut protégé, - un attribut privé, + un membre public.

Question 4 : Implémentation des classes en C++

Le code complet (fichier exo3_personnel.cpp) est donné ci-dessous. Il a été compilé avec g++ -std=c++17 et testé.

```
#include <iomanip>
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Employe { // classe abstraite
protected:
    string matricule, nom, prenom;     double salaireBase;
public:     Employe(string m, string n, string p, double s)
: matricule(m), nom(n), prenom(p), salaireBase(s) {}
virtual ~Employe() = default;
    virtual double calculerSalaire() const = 0;   // méthode virtuelle pure
virtual string categorie() const = 0;     virtual void afficher() const {
        cout << categorie() << " | " << matricule << " | " << prenom << " " << nom
<< " | base : " << salaireBase << " | salaire : " << calculerSalaire() << endl;
    }
};

class Enseignant : public Employe {
    double indemniteRecherche, indemniteEnseignement;
public:
    Enseignant(string m, string n, string p, double s, double ir, double ie)
: Employe(m, n, p, s), indemniteRecherche(ir), indemniteEnseignement(ie) {}
double calculerSalaire() const override {         return salaireBase +
```

```
indemniteRecherche + indemniteEnseignement;     }     string categorie() const
override { return "Enseignant"; } };

class AgentAdministratif : public Employe {     double primeResponsabilite; public:
AgentAdministratif(string m, string n, string p, double s, double pr)         : Employe(m,
n, p, s), primeResponsabilite(pr) {}     double calculerSalaire() const override { return
salaireBase + primeResponsabilite; }     string categorie() const override { return "Agent
administratif"; } };

class Technicien : public Employe {     double primeRisque, primeAstreinte; public:
Technicien(string m, string n, string p, double s, double pr, double pa)         : Employe(m, n, p,
s), primeRisque(pr), primeAstreinte(pa) {}     double calculerSalaire() const override { return
salaireBase + primeRisque + primeAstreinte; }     string categorie() const override { return
"Technicien"; } };

int main() {
    cout << fixed << setprecision(0);     vector<unique_ptr<Employe>> personnel;
personnel.push_back(make_unique<Enseignant>("EN01", "Houngbo", "Marc", 400000, 80000, 50000));
personnel.push_back(make_unique<AgentAdministratif>("AD01", "Sagbo", "Rita", 250000, 30000));
personnel.push_back(make_unique<Technicien>("TE01", "Adjovi", "Paul", 220000, 25000, 15000));

    double masse = 0;     for (const auto& e : personnel) {
e->afficher();                 // liaison dynamique
masse += e->calculerSalaire();     }     cout << "Masse
salariale totale : " << masse << endl; }
```

Question 5 : Utiliser les méthodes virtuelles pour calculer le salaire

La classe mère déclare la méthode virtuelle pure, et chaque classe dérivée la redéfinit avec le mot-clé override :

```
// Dans Employe virtual double
calculerSalaire() const = 0;

// Dans Enseignant double
calculerSalaire() const override {
    return salaireBase + indemniteRecherche + indemniteEnseignement; }
// Dans AgentAdministratif : salaireBase + primeResponsabilite
// Dans Technicien         : salaireBase + primeRisque + primeAstreinte
```

Catégorie	Formule du salaire
Enseignant	base + indemnité recherche + indemnité enseignement
Agent administratif	base + prime de responsabilité
Technicien	base + prime de risque + prime d'astreinte

Question 6 : Afficher les informations de chaque employé

La méthode afficher() est définie une seule fois dans Employe. Elle appelle categorie() et calculerSalaire(), deux méthodes virtuelles : chaque employé affiche donc ses informations avec sa propre catégorie et son propre salaire. `virtual void afficher() const {`

```
    cout << categorie() << " | " << matricule << " | " << prenom << " " << nom
<< " | base : " << salaireBase << " | salaire : " << calculerSalaire() << endl;
}
```

Question 7 : Créer une collection contenant plusieurs catégories d'employés

On utilise un vector de pointeurs intelligents vers la classe mère. Chaque élément peut être un objet d'une classe dérivée différente. La liaison dynamique choisit la bonne version de calculerSalaire() à l'exécution. Le destructeur virtuel de Employe garantit une destruction correcte.

```
vector<unique_ptr<Employe>> personnel; personnel.push_back(make_unique<Enseignant>("EN01",
"Houngbo", "Marc", 400000, 80000, 50000));
```

```
personnel.push_back(make_unique<AgentAdministratif>("AD01", "Sagbo", "Rita", 250000, 30000));
personnel.push_back(make_unique<Technicien>("TE01", "Adjovi", "Paul", 220000, 25000, 15000));

double masse = 0; for (const auto& e : personnel) {
e->afficher();                    // liaison dynamique
masse += e->calculerSalaire(); }
```

Résultat de l'exécution :

```
Enseignant | EN01 | Marc Houngbo | base : 400000 | salaire : 530000
Agent administratif | AD01 | Rita Sagbo | base : 250000 | salaire : 280000
Technicien | TE01 | Paul Adjovi | base : 220000 | salaire : 260000
Masse salariale totale : 1070000
```

Notions mobilisées

- **Héritage** : class Enseignant : public Employe.
- **Classe abstraite** : Employe contient une méthode virtuelle pure.
- **Méthodes virtuelles et redéfinition** : virtual dans la mère, override dans les filles.
- **Polymorphisme et liaison dynamique** : un même appel e->calculerSalaire() donne un résultat différent selon le type réel de l'objet.